

Bulletproofs*: Verifier-Efficient Arithmetic Circuit Proofs via Folding

Emanuele Scala¹ and Daniele Bartoli²

¹ emsczpk@proton.me

² Department of Mathematics and Computer Science, University of Perugia

Abstract. We present *Bulletproofs** (BP*, BulletproofsStar), a folding scheme for arithmetic circuit proofs under standard assumptions and without preprocessing, i.e., for the arithmetic circuit satisfiability language of Bulletproofs (S&P 18), following the recipe of ProtoStar (ePrint 2023/620). To this end, we first adapt the algebraic verifiers of the arithmetic circuit proof of Bulletproofs to the algebraic form required by ProtoStar, and prove that the modified protocol remains secure. Then, we design the *Bulletproofs** folding scheme that is complete and knowledge-sound. Finally, we analyze the resulting verifier cost after the folding-to-IVC transformation. The result shows an asymptotic linear gain compared to repeated invocations of the monolithic Bulletproofs verifier.

Keywords: Folding Scheme · Bulletproofs · ProtoStar · Zero-Knowledge Proofs · Incremental Verifiable Computation · Recursive Proofs.

1 Introduction

Folding schemes are the building blocks of Incrementally Verifiable Computations (IVCs): proofs that attest to the correct execution of a computation together with the validity of a previous proof [8].

Prior to folding schemes, recursive proofs realize IVCs where the expensive work of the verifier circuit is treated as part of the statement, by taking as public inputs both the input and the prover’s claimed output. This allows the verifier to merge two instances of the statement into a single instance, thus amortizing the cost of the expensive work across many composed proofs. Finally, only at the end of the recursion, the ultimate verifier executes the expensive check to validate the entire sequence of proofs. However, this approach still requires the verifier circuit to partially verify a proof at each recursion step [18].

With folding schemes one can check two instances in some relation by checking a single (folded) instance. Then, instead of accumulating expensive steps, an IVC based on folding schemes accumulates instances themselves. Concretely, at each incremental step, the IVC prover proves that it has executed the step computation and that it has folded its prior step into a running relaxed instance, while the verifier circuit verifies that the prover correctly folds the previous step. A folding scheme is non-trivial when the resulting verifier time and communication are lower than a verifier checking each original instance separately [18].

ProtoStar [11] allows to build secure and efficient folding schemes from interactive arguments whose verifier is of special form. ProtoStar recipe involves the following ingredients: i) use succinct homomorphic commitment scheme; ii) apply Fiat-Shamir transform to obtain a non-interactive argument of knowledge (NARK); iii) build an accumulation scheme, where the accumulator is the NARK proof, by sampling a random challenge and taking a random linear combination of the current accumulator with the new proof.

In ProtoStar, the accumulated instances are the "relaxed" verifier checks of the NARK augmented with error terms that arise at each accumulation step: given two NARK proofs, the prover folds them via a random linear combination and generates a list of d error terms (with d the degree of the NARK verifier) as an accumulation proof. The verifier only needs to check that the accumulated instances are consistent with the accumulation proof and the original instances being folded. After all accumulation steps, a decider performs a final check to verify the consistency of commitments in the accumulator and that the "relaxed" NARK verifier check still passes. An accepting decider implies that all accumulated NARKs were valid. Finally, a folding-to-IVC transformation can be done by invoking the compiler of [12] which yields a secure IVC scheme.

1.1 Contribution

The purpose of this paper is to improve the verification efficiency of Bulletproofs-based Zero-Knowledge Proofs (ZKPs). This is particularly beneficial for confidential transaction protocols that rely on such trustless ZKP primitives [1, 16, 9, 22]. In a previous work, Bulletproofs+ [14] (BP+), a weighed Inner-Product Argument (IPA) is used to reduce the proof size, while maintaining verifier time comparable to Bulletproofs. Bulletproofs++ [15] (BP++) builds on BP+ and introduces a weighed norm argument, improving verifier time by a logarithmic factor. Other works, such as the Compressed Σ -IPA/protocol [2, 23] yield a lighter version of the IPA with fewer cryptographic operations. However, these approaches still work for the monolithic Bulletproofs verification. An attempt to apply recursive proofs to Bulletproofs-style arguments appears in Halo [8], which achieves amortized verifier time through proof batching. However, Halo modifies the IPA relation, resulting in a new polynomial evaluation proof, and is tied to the Sonic [20] proof system. In contrast, we apply the generic ProtoStar folding recipe without modifying the original Bulletproofs relation. Moreover, we enable a folding scheme for the arithmetic circuit satisfiability language of Bulletproofs, thus obtaining a construction under standard assumptions and without preprocessing. To achieve this, we slightly modify the arithmetic circuit proof protocol to cast its algebraic verifiers into the form required by ProtoStar. We then design the *Bulletproofs** (BP*, BulletproofsStar) folding scheme that is complete and knowledge-sound. Finally, by applying the folding-to-IVC transformation of [12, 11], we estimate the verifier cost of the resulting IVC. This yields an asymptotic linear gain compared to repeated invocations of the monolithic Bulletproofs verifier.

1.2 Organization

Section 2 provides notation, security definitions, and cryptographic primitives. Section 3 is the related work on recursive proofs and folding schemes. Section 4 presents our folding scheme design, introduces the modified Bulletproofs arithmetic circuit proof, and derives the error terms required for folding; Section 5 presents the *Bulletproofs** folding scheme and analyzes its complexity; Section 6 evaluates *Bulletproofs** against the reference proof system; Section 7 gives the conclusions and future directions.

2 Preliminaries

2.1 Notations

With PPT we specify probabilistic polynomial-time algorithms and with λ the security parameter. We write $x \xleftarrow{\$} \mathcal{X}$ when we sample x uniformly at random from the set $\mathcal{X}$. We denote by $\mathbb{G}$ a cyclic group of prime order p . We consider G a generator of $\mathbb{G}$, $H \xleftarrow{\$} \mathbb{G}$ a distinct generator from G . We denote by $\mathbb{Z}_p$ the ring of integers modulo p and by $\mathbb{Z}_p^*$ the set of its invertible elements. We use additive notation for group operations in $\mathbb{G}$: the addition of two elements $A, B \in \mathbb{G}$ is $A + B$, the scalar multiplication is defined as $k \cdot A$ for any $k \in \mathbb{Z}_p$. We write $\mathbf{a} = (a_1, \dots, a_n) \in \mathbb{Z}_p^n$ for n -length vectors of scalars and $\mathbf{G} = (G_1, \dots, G_n) \in \mathbb{G}^n$ for n -length vectors of generators. The inner product between two n -length scalar vectors $\mathbf{a}$ and $\mathbf{b}$ is $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^n a_i \cdot b_i \in \mathbb{Z}_p$, the Hadamard product between the same vectors is $\mathbf{a} \circ \mathbf{b} = (a_1 \cdot b_1, \dots, a_n \cdot b_n) \in \mathbb{Z}_p^n$. Let $\mathbf{a} \in \mathbb{Z}_p^n$ and $\mathbf{G} \in \mathbb{G}^n$, the multi-scalar multiplication is defined as $\mathbf{a} \cdot \mathbf{G} = \langle \mathbf{a}, \mathbf{G} \rangle \in \mathbb{G}$. We write $\mathbf{A} \in \mathbb{Z}_p^{m \times n}$ for matrices with m rows and n columns and $a_{i,j}$ for elements of $\mathbf{A}$ in the i -th row and j -th column. Let $\mathbf{A} \in \mathbb{Z}_p^{m \times n}$, $\mathbf{a} \in \mathbb{Z}_p^n$ and $\mathbf{b} \in \mathbb{Z}_p^m$, we define $\mathbf{A} \cdot \mathbf{a} = (\sum_{j=1}^n w_{1,j} a_j, \dots, \sum_{j=1}^n w_{m,j} a_j) \in \mathbb{Z}_p^m$ and $\mathbf{b} \cdot \mathbf{A} = (\sum_{i=1}^m b_i w_{i,1}, \dots, \sum_{i=1}^m b_i w_{i,n}) \in \mathbb{Z}_p^n$. We denote by $\mathbf{a}_{[k]} = (a_1, \dots, a_k) \in \mathbb{Z}_p^k$ and $\mathbf{a}_{[k:]} = (a_{k+1}, \dots, a_n) \in \mathbb{Z}_p^{n-k}$ the slices of $\mathbf{a} \in \mathbb{Z}_p^n$.

2.2 Special-sound arguments of knowledge

We revise the following definitions from [2, 24, 11, 21].

Definition 1 (Public-coin interactive arguments [11]). Let $\mathcal{R}$ be a relation for statement x and witness w , defining an NP language $\mathcal{L}$. In particular, $x \in \mathcal{L}$ implies that there exists an efficiently verifiable witness w such that $(x, w) \in \mathcal{R}$. An interactive argument for $\mathcal{R}$ is a two-party conversation between two PPT algorithms, a prover $\mathcal{P}$ and a verifier $\mathcal{V}$, with $\mathcal{P}$ and $\mathcal{V}$ taking as input the public statement x and only $\mathcal{P}$ the private witness w . The conversation produces a transcript $tr \leftarrow \langle \mathcal{P}(x, w), \mathcal{V}(x) \rangle$. We say that the transcript is accepting if $\mathcal{V}$ outputs 1 after interacting with $\mathcal{P}$, and we denote this event by $\langle \mathcal{P}(x, w), \mathcal{V}(x) \rangle = 1$, rejecting otherwise. The acceptance is interpreted as $\mathcal{V}$ being convinced that $x \in \mathcal{L}$. In

an interactive argument, completeness ensures that $\mathcal{V}$ accepts true statements, whereas soundness ensures that the verifier rejects false statements from any PPT prover $\mathcal{P}^*$ except with negligible probability. We say that an interactive argument is public-coin if the messages of $\mathcal{V}$ are only random values generated independently of the messages of $\mathcal{P}$. An interactive argument is $(2k + 1)$ -move if there are k verifier messages and $k + 1$ prover messages.

Definition 2 (Interactive arguments of knowledge). A public-coin interactive argument is an argument of knowledge for a relation $\mathcal{R}$ if it has perfect completeness and special-soundness.

Definition 3 (Perfect completeness [24]). An interactive argument for a relation $\mathcal{R}$ is perfectly complete if the verifier always accepts a conversation with a prover acting honestly.

Definition 4 (t-Special-Soundness [2]). A 3-move public-coin interactive argument for a relation $\mathcal{R}$ is t -special-sound if there exists a polynomial-time extractor $\mathcal{E}$ such that, given the statement x and t accepting transcripts $(a, c_1, z_1), \dots, (a, c_t, z_t)$ with the same prover's first message a and pairwise-distinct verifier's challenges $c_1, \dots, c_t$, the extractor outputs a witness w such that $(x, w) \in \mathcal{R}$. When $t = 2$ we simply say that the argument is special-sound.

A generalization of t -special-soundness for $(2k + 1)$ -move public-coin interactive arguments $(\mathcal{P}, \mathcal{V})$ is known as $(t_1, \dots, t_k)$ -special-soundness [7]. In this version, the polynomial-time extractor $\mathcal{E}$ takes as input the statement x and a $(t_1, \dots, t_k)$ -tree of accepting transcripts as in Definition 5, and outputs a witness satisfying $(x, w) \in \mathcal{R}$. Moreover, if such an efficient extraction procedure exists from a tree of transcripts with this structure, then $(\mathcal{P}, \mathcal{V})$ has *witness-extended emulation* by the forking lemma [19].

Definition 5 (Tree of transcripts [2]). A $(t_1, \dots, t_k)$ -tree of transcripts for a $(2k + 1)$ -move public-coin interactive argument is a set of $T = \prod_{i=1}^k t_i$ transcripts organized in a tree structure. In this structure, each node corresponds to a prover's message (which may be empty) and each edge is labeled with a verifier's challenge. A node at depth i has precisely t_i children, each linked to a pairwise distinct challenge. The full tree contains T unique paths from root to leaf, with each path corresponding to an accepting transcript.

2.3 Zero-knowledge arguments and Fiat-Shamir transform

An additional property of arguments of knowledge is the *zero-knowledge*. In this setting, the prover $\mathcal{P}$ aims to convince the verifier $\mathcal{V}$ of a statement's validity without revealing any additional information, particularly about the secret witness. The definition of zero-knowledge follows the simulation paradigm. We use the specific notion of *Special Honest-Verifier Zero-Knowledge* (SHVZK), that requires a simulator for the honest verifier and with the verifier's internal randomness known to the simulator in advance [10].

Definition 6 (Perfect Special Honest-Verifier Zero-Knowledge [24]).

A public-coin argument of knowledge $(\mathcal{P}, \mathcal{V})$ has perfect honest-verifier zero-knowledge if there exists a PPT algorithm $\mathcal{S}$, known as simulator, which, given as input the public statement x and the public-coin randomness of $\mathcal{V}$, produces transcripts whose distribution is the same as that of transcripts generated during the protocol executions of $(\mathcal{P}, \mathcal{V})$.

We can convert a public-coin argument of knowledge into a Non-Interactive Argument of Knowledge (NARK) using the Fiat-Shamir transform [17]. This transformation replaces each random challenge with the hash of the transcript up to that point, thus yielding a NARK in the random oracle model [4]. The prover and verifier of a random-oracle NARK, i.e., $(\mathcal{P}^{\mathcal{O}_{NARK}}, \mathcal{V}^{\mathcal{O}_{NARK}})$, are called *T-query random-oracle algorithms* when they are given black-box access to a random oracle $\mathcal{O}_{NARK}$ and make at most T queries to it. The knowledge-soundness property for random-oracle NARKs necessarily depends on the query complexity T .

Definition 7 (Knowledge-Soundness [3]). A random-oracle NARK $(\mathcal{P}^{\mathcal{O}_{NARK}}, \mathcal{V}^{\mathcal{O}_{NARK}})$ is knowledge-sound with knowledge error $\mu : \mathbb{N} \times \mathbb{N} \rightarrow \{0, 1\}$, if there exists a knowledge extractor $\mathcal{E}$ with the following properties: $\mathcal{E}^{\mathcal{P}^*}(x)$, with input statement x and oracle access to a (potentially dishonest) T -query random-oracle prover $\mathcal{P}^*$, runs in an expected polynomial time in $|x|$ and T and outputs a witness w such that $(x, w) \in \mathcal{R}$ with probability

$$\Pr[(x, w) \in \mathcal{R} : w \leftarrow \mathcal{E}^{\mathcal{P}^*}(x)] \geq \frac{\epsilon(\mathcal{P}^*, x) - \mu(|x|, T)}{\text{poly}(|x|)},$$

where $\epsilon(\mathcal{P}^*, x) = \Pr[\mathcal{V}^{\mathcal{O}_{NARK}}(x, \pi) = 1 : \pi \leftarrow \mathcal{P}^*]$. Here, $\mathcal{E}$ implements $\mathcal{O}_{NARK}$ for $\mathcal{P}^*$, in particular, $\mathcal{E}$ can arbitrarily program $\mathcal{O}_{NARK}$.

Adaptive knowledge-soundness for random-oracle NARKs differs from the non-adaptive knowledge-soundness in Definition 7 in that $\mathcal{P}^*$ outputs not only a proof π but also a statement x , i.e., $\mathcal{P}^*$ is free to choose x adaptively. In this setting, the extractor is required to output $(x, \pi, v; w)$, where $v \in \{0, 1\}$ indicates whether π is a valid proof for x and (x, π, v) is identically distributed to $(x, \pi, v) \leftarrow \mathcal{P}^*$. Then, the success probability of the extractor depends on the success probability $\epsilon(\mathcal{P}^*)$ of the adaptive prover $\mathcal{P}^*$. To support compositional settings, $\mathcal{P}^*$ may also output auxiliary information aux , while the extractor is required to simulate the tuple (x, π, aux, v) [3].

The Fiat-Shamir transformation is adaptive when the statement x is also hashed with the transcript (e.g., it may be included in the first challenge). A positive result in [3] establishes an upper bound on the knowledge error of Fiat-Shamir transformed $(2k + 1)$ -move public-coin interactive arguments.

Lemma 1 (Fiat-Shamir transform of Special-Sound Arguments [11]).

The Fiat-Shamir transform of a $(t_1, \dots, t_k)$ -out-of- N , with N -sized challenge

space, *special-sound interactive argument is knowledge sound with knowledge error*

$$\mu_{fs}(T) = (T + 1)\mu$$

where $\mu = 1 - \prod(1 - \frac{t_i}{N})$ is the knowledge error of the interactive argument.

2.4 Assumptions and commitments

We work in prime-order cyclic groups where the discrete logarithm problem is assumed to be computationally intractable (DLOG assumption).

Definition 8 (Non-trivial DLOG relation [10]). *For any PPT adversary $\mathcal{A}$ and any $j \geq 2$, there exists a negligible function $\text{negl}(\lambda)$, such that*

$$\Pr \left[\begin{array}{l} G_1, \dots, G_j \xleftarrow{\$} \mathbb{G}; \quad b_1, \dots, b_j \in \mathbb{Z}_p \leftarrow \mathcal{A}(\mathbb{G}, G_1, \dots, G_j) \\ : \quad b_1, \dots, b_j \neq 0 \wedge \sum_{i=1}^j b_i G_i = 0 \end{array} \right] \leq \text{negl}(\lambda)$$

In other words, the probability that $\mathcal{A}$ finds a non-trivial DLOG relation between uniformly random group elements is negligible.

We use *Pedersen commitments* [24] with a commitment space $\mathbb{G}$, and a message space $\mathbb{Z}_p$. Let $G, H \in \mathbb{G}$, $\mathbf{G} \in \mathbb{G}^n$, $m \in \mathbb{Z}_p$ and $r \xleftarrow{\$} \mathbb{Z}_p^*$ (a random blinding term). A commitment C to m is computed as

$$C = mG + rH \in \mathbb{G}.$$

If $\mathbf{m} \in \mathbb{Z}_p^n$, a vector commitment C to $\mathbf{m}$ is computed as

$$C = \langle \mathbf{m}, \mathbf{G} \rangle + rH \in \mathbb{G}.$$

Let $C_1, C_2 \in \mathbb{G}$ be two commitments to $m_1, m_2 \in \mathbb{Z}_p$ respectively, and $r_1, r_2 \xleftarrow{\$} \mathbb{Z}_p^*$. Pedersen commitments are *homomorphic*, thus we can compute

$$C_1 + C_2 = (m_1 + m_2)G + (r_1 + r_2)H \in \mathbb{G}.$$

Pedersen commitments are *binding* and *hiding* under the DLOG assumption. Binding means that it is computationally hard to find two openings for a commitment. Hiding means that it is hard to distinguish which is the committed message. If no blinding term is used, the resulting commitment is binding but not hiding.

We sometimes write a commitment function

$$\text{Commit}(\mathbf{m}_1, \mathbf{m}_2, r) := \langle \mathbf{m}_1, \mathbf{G} \rangle + \langle \mathbf{m}_2, \mathbf{H} \rangle + rH,$$

that maps to the relative generator (or vector of generators) according to the input.

2.5 Folding schemes

We recall the notion of folding schemes from [11], which ProtoStar also refers to as accumulation schemes. In particular, what gets folded are two NARK proofs: one associated with the running instance and one carried by an accumulator. Accordingly, a folding scheme for a NARK $(\mathcal{P}_{NARK}, \mathcal{V}_{NARK})$ is a triple of algorithms, prover, verifier, and decider $(\mathcal{P}_{fold}, \mathcal{V}_{fold}, \mathcal{D})$, all with access to the same random oracles $\mathcal{O}_{NARK}, \mathcal{O}_{fold}$:

- $\mathcal{P}_{fold}(\text{pi}, \pi = (\pi_x, \pi_w), \text{acc} = (\text{acc}_x, \text{acc}_w)) \rightarrow \{\text{acc}' = (\text{acc}'_x, \text{acc}'_w), \text{pf}\}$, where pi is the statement, π the NARK proof, acc the accumulator, acc' the new accumulator, pf the accumulation proof.
- $\mathcal{V}_{fold}(\text{pi}, \pi_x, \text{acc}_x, \text{acc}'_x, \text{pf}) \rightarrow \{0, 1\}$, where pi is the statement, π_x the NARK proof instance, acc_x and acc'_x the old and new accumulator instances, pf the accumulation proof. The verifier outputs 1 (accept) or 0 (reject).
- $\mathcal{D}(\text{acc}) \rightarrow \{0, 1\}$. The decider on input the accumulator outputs 1 (accept) or 0 (reject).

A folding scheme is perfectly complete and knowledge-sound with knowledge error μ if the random-oracle NARK $(\mathcal{P}', \mathcal{V}')$ is perfectly complete and knowledge-sound with knowledge error μ for the relation

$$\mathcal{R}_{fold} = \{(\text{pi}, \pi_x, \text{acc}_x; \pi_w, \text{acc}_w) : \mathcal{V}_{NARK}(\text{pi}, \pi) = 1 \wedge \mathcal{D}(\text{acc}) = 1\}. \quad (1)$$

where $\mathcal{P}'$ outputs the tuple (acc', pf) and $\mathcal{V}'$, on input $((\text{pi}, \pi_x, \text{acc}_x), (\text{acc}', \text{pf}))$, accepts if $\mathcal{D}(\text{acc}') = 1$ and $\mathcal{V}_{fold}(\text{pi}, \pi_x, \text{acc}_x, \text{acc}'_x, \text{pf}) = 1$.

2.6 Bulletproofs arithmetic circuits

We recall the arithmetic circuit relation of Bulletproofs [10]:

$$\begin{aligned} \mathcal{R}_{AC} = \{ & (G, H \in \mathbb{G}, \mathbf{G}, \mathbf{H} \in \mathbb{G}^n, \mathbf{V} \in \mathbb{G}^m, \mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O \in \mathbb{Z}_p^{Q \times n}, \\ & \mathbf{W}_V \in \mathbb{Z}_p^{Q \times m}, \mathbf{c} \in \mathbb{Z}_p^Q; \mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O \in \mathbb{Z}_p^n, \mathbf{v}, \gamma \in \mathbb{Z}_p^m) : \\ & V_j = v_j G + \gamma_j H \ \forall j \in [1, m] \wedge \mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O \wedge \\ & \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c} \}. \end{aligned} \quad (2)$$

Hence, an arithmetic circuit with n multiplication gates can be converted into a Hadamard-product relation with additional $Q \leq 2n$ linear constraints that include the opening of commitments V_j .

3 Related Work

Eli Ben-Sasson et al. [5] implement the first scalable preprocessing (trusted setup) zk-SNARK that is practical for recursive proof composition by exploiting cycles of elliptic curves. Such result highlights how suitably combined elliptic

curves lie on a PCD-friendly³ cycle; a known example is the PCD-friendly 2-cycle of two elliptic curves where the base field size of one curve equals the group order of the other curve (and viceversa). In this setting, it is possible to construct two zk-SNARKs on the two curves, and use them to enable efficient recursion. In particular, each zk-SNARK prove statements about the verifier of the other zk-SNARK. By alternating the two proof systems, each verification step is expressed in the field that the other system can handle natively.

Halo [8] enables recursive proof composition without a trusted setup under standard assumptions and over cycles of elliptic curves that are not pairing-friendly. First, Halo presents a Polynomial Commitment Scheme (PCS) based on a modified Inner Product Argument (IPA) relation of [7, 10] that is tied to a variant of Sonic [20] proof system to work with the PCS. The resulting argument for arithmetic circuit satisfiability exploits polynomial commitment amortization techniques, and a nested amortization strategy is used to enable recursive proof composition. The verification circuit avoids performing any linear-time work, instead, it treats the deferred check as part of the statement by taking as public inputs both the input and the prover’s claimed output of the linear-time computation. This allows the verifier to merge two instances of the statement into a single instance, thus amortizing the cost of the expensive operation across many composed proofs. Finally, only at the end of the recursion, the ultimate verifier executes the linear-time check to validate the entire sequence of proofs.

Proof-Carrying Data (PCD) [13] builds on the previous result and shows that there is a general paradigm to construct *accumulation schemes* for succinct non-interactive arguments of knowledge (SNARKs) even with non-sublinear-time verifier. In particular, an accumulation scheme can be derived for any SNARK whose verifier is succinct except for the evaluation of a specific statement. Moreover, if the specific statements are opening of polynomial commitments, then two PCS, one based on DLOG assumptions and the other based on knowledge assumptions in bilinear groups, have accumulation schemes in the random oracle model. Finally, the authors prove that any SNARK with a sublinear accumulation verifier yields a PCD scheme.

Halo Infinite [6] further generalizes the previous result, showing that PCD can be built heuristically from any homomorphic PCS, even when evaluation proofs are not sublinear in either proof size (w.r.t. the polynomial degree) or verification time. In particular, the authors observe that Halo’s approach applies to any PCS that supports linear combinations of commitments, which defines a Linear Combination Scheme (LCS). Moreover, the authors outline the potential of a PCS to aggregate evaluation proofs to amortize the verifier work. This lands to two generalized results: i) every PCS with efficient LCS has an efficient private aggregation, i.e., an interactive proof that combines many evaluation proof instances into one such that accepting the aggregate implies validity of all instances; ii) every succinct additive PCS has an efficient public aggregation, i.e., many evaluation proofs can be aggregated non-interactively into one instance,

³ Since they facilitate the construction of PCD, i.e., a primitive that encapsulates the recursive proof composition.

even without knowing witnesses for the input instances, and a valid proof for the output instance implies validity of each of the input proofs. Overall, aggregation enables batch evaluation of PCS and yields generalized recursive proofs/PCD constructions: PCD with an efficient prover can be obtained from PCS admitting an efficient LCS, PCD with sublinear proofs follows from PCS with an efficient public aggregation.

PCD "without succinct arguments" [12] shows that a PCD can be constructed from any non-interactive argument of knowledge (NARK) that satisfies the notion of *split accumulation*. A split accumulator divides a classical accumulator into an instance part and a witness part, and the accumulation verifier only receives the instance part. This enables to design a succinct accumulation verifier with short instances and not large witnesses. Building on this notion, the authors show that any NARK having a split accumulation with a sublinear accumulation verifier yields a PCD, even if NARK does not have sublinear argument size. Focusing on minimizing the cost of the accumulation verifier, the authors develop split accumulation for a transparent NARK for R1CS and for predicates such as Hadamard products and polynomial evaluations based on Pedersen commitments.

Nova [18] introduces a (trustless and DLOG-based) primitive called *folding scheme*, which can be used to reduce the task of checking two instances in some relation to the task of checking a single (folded) instance. Unlike prior SNARK-based IVC approaches, which accumulate expensive steps of the verifier circuit, Nova accumulates the instances themselves. Concretely, Nova instantiates a folding scheme for arithmetic circuit satisfiability by introducing *relaxed R1CS* and relying on succinct homomorphic commitments (e.g., Pedersen commitments). Nova then builds an IVC from a non-interactive folding scheme for relaxed R1CS: all steps of the incremental computation are expressed with R1CS (sharing the same coefficient matrices), and at each step Nova prover proves that it has executed the step computation and has folded its prior step into a running relaxed R1CS instance. In the incremental steps, the (recursively composed) verifier circuit verifies the prover correctly folds the previous step. The resulting IVC proof size and verifier time are linear in the size of the computation, but they can be optimized using zkSNARKs.

4 Bulletproofs* design

In this section, we give an overview of ProtoStar, describe how we adapt the algebraic verifier of Bulletproofs to the ProtoStar form, and show how our prover computes the error terms required by the folding scheme design.

4.1 ProtoStar and negative exponents

ProtoStar works for public-coin $(2k+1)$ -move interactive arguments $(\mathcal{P}, \mathcal{V})$ whose verifier, after k challenges, computes an algebraic map $\mathcal{V}$ of degree D and checks that the output is a zero vector of length l . For a transcript $((\mathbf{m}_i, r_i)_{i=1}^k, \mathbf{m}_{k+1})$

of prover messages $(\mathbf{m}_i)_{i=1}^{k+1}$ and verifier challenges $(r_i)_{i=1}^k$, the map $\mathcal{V}$ is defined as the sum of $D + 1$ homogeneous degree- d algebraic maps:

$$\mathcal{V}(\text{pi}, \mathbf{m}_1, \dots, \mathbf{m}_{k+1}, \mathbf{r}) := \sum_{d=0}^D f_d^{\mathcal{V}}(\text{pi}, \mathbf{m}_1, \dots, \mathbf{m}_{k+1}, \mathbf{r}) = \mathbf{0}. \quad (3)$$

Let $(\mathcal{P}_{NARK}, \mathcal{V}_{NARK})$ be the NARK of the $(2k + 1)$ -move interactive argument $(\mathcal{P}, \mathcal{V})$, where the verifier challenges are derived via a random oracle, and π the NARK proof. The "relaxed" verifier checks introduce a slack variable σ and a vector of error terms $\mathbf{e}$ of length l :

$$\mathcal{V}(\text{pi}, \pi, \mathbf{m}_1, \dots, \pi, \mathbf{m}_{k+1}, \mathbf{r}, \sigma) := \sum_{d=0}^D \sigma^{D-d} \cdot f_d^{\mathcal{V}}(\text{pi}, \pi, \mathbf{m}_1, \dots, \pi, \mathbf{m}_{k+1}, \mathbf{r}) = \mathbf{e}. \quad (4)$$

Then, in the folding scheme the prover accumulates "relaxed" verifier checks of the NARK by computing:

$$\sum_{d=0}^D (X + \sigma)^{D-d} \cdot f_d^{\mathcal{V}}(X \cdot \text{pi} + \text{pi}', (X \cdot \mathbf{m}_i + \mathbf{m}'_i)_{i=1}^{k+1}, (X \cdot r_i + r'_i)_{i=1}^k), \quad (5)$$

which interpolates statement and NARK proof with the old accumulator and brings up the error terms that are used as an accumulation proof. Indeed, Equation (5) turns out to be, with $\mathcal{V}_{NARK} = 0$, equal to:

$$\mathbf{e} + \sum_{j=1}^{D-1} \mathbf{e}_j X^j + \mathcal{V}_{NARK}(\text{pi}, (\mathbf{m}_i)_{i=1}^{k+1}, (r_i)_{i=1}^k) X^D. \quad (6)$$

The calculation of error terms is a critical step in the design of a folding scheme, since the verifier's checks must have a special form, i.e., $\mathcal{V}$ is the sum of $D + 1$ homogeneous degree- d algebraic maps. In any public-coin $(2k + 1)$ -move interactive argument with an algebraic verifier, when $\mathcal{V}$ checks equations with negative exponents, then the accumulated instances cannot be computed, as illustrated by the following example. Consider the map $f(r) = r^{-1}$, this results $f(Xr + r') = (Xr + r')^{-1}$, which does not imply an algebraic $\mathcal{V}$ of the right form $a_0 + a_1 X + \dots + a_d X^d$ for any finite degree d and coefficients $(a_i)_{i=0}^d$.

4.2 Modified arithmetic circuit proof and homogeneous maps

We introduce a modified arithmetic circuit proof of Bulletproofs satisfying Relation (2). At a high level, the arithmetic circuit proof of Bulletproofs reduces the Hadamard-product relation, together with the linear constraints, to a single inner-product relation. Our protocol follows from the argument of Bulletproofs, except that we replace the verifier check enforcing the constraints with an equivalent check that avoids negative exponents. The full modified interactive argument $(\mathcal{P}, \mathcal{V})$ is shown in Figure 1.

-
- 1: $(\mathcal{P}, \mathcal{V})$'s **input**: $(G, H, \mathbf{G}, \mathbf{H}, \mathbf{V}, \mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O, \mathbf{W}_V, \mathbf{c}; \mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O, \mathbf{v}, \gamma)$
 - 2: $\mathcal{V}$'s **output**: $\{0, 1\}$
 - 3: $\mathcal{P}$:
 - 4: $\gamma_I, \gamma_O, \gamma_S \xleftarrow{\$} \mathbb{Z}_p$
 - 5: $\mathbf{s}_L, \mathbf{s}_R \xleftarrow{\$} \mathbb{Z}_p^n$
 - 6: $A_I = \gamma_I H + \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle$
 - 7: $A_O = \gamma_O H + \langle \mathbf{a}_O, \mathbf{G} \rangle$
 - 8: $S = \gamma_S H + \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle$
 - 9: $\mathcal{P} \rightarrow \mathcal{V} : A_I, A_O, S$
 - 10: $\mathcal{V} \rightarrow \mathcal{P} : y, z \xleftarrow{\$} \mathbb{Z}_p^*$
 - 11: $\mathcal{P}$ and $\mathcal{V}$:
 - 12: $\mathbf{z}_{[1:]}^{Q+1} = (z, z^2, \dots, z^Q)$
 - 13: $\mathbf{y}^n = (1, y, y^2, \dots, y^{n-1})$
 - 14: $\delta_{new}(y, z) = \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R), \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L \rangle$
 - 15: $\mathcal{P}$:
 - 16: $l(X) = X(\mathbf{a}_L + \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R)) + X^2 \mathbf{a}_O + X^3 \mathbf{s}_L$
 - 17: $r(X) = -\mathbf{y}^{-n} + \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_O + X(\mathbf{y}^{-n} \circ \mathbf{a}_R + \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L) + X^3(\mathbf{y}^{-n} \circ \mathbf{s}_R)$
 - 18: $t(X) = \langle l(X), r(X) \rangle$
 - 19: $w = \langle \mathbf{a}_L, \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L \rangle + \langle \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R, \mathbf{a}_R \rangle + \langle \mathbf{a}_O, \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_O \rangle$
 - 20: $t_2 = \langle \mathbf{a}_L, \mathbf{y}^{-n} \circ \mathbf{a}_R \rangle - \langle \mathbf{a}_O, \mathbf{y}^{-n} \rangle + w + \delta_{new}(y, z)$
 - 21: $\rho_{1,3,4,5,6} \xleftarrow{\$} \mathbb{Z}_p$
 - 22: $T_{1,3,4,5,6} \leftarrow t_{1,3,4,5,6} G + \rho_{1,3,4,5,6} H$
 - 23: $\mathcal{P} \rightarrow \mathcal{V} : T_1, T_3, T_4, T_5, T_6$
 - 24: $\mathcal{V} \rightarrow \mathcal{P} : x \xleftarrow{\$} \mathbb{Z}_p^*$
 - 25: $\mathcal{P}$:
 - 26: $\tau_x = \rho_1 x + \sum_{i=3}^6 \rho_i x^i + x^2 \langle \mathbf{z}_{[1:]}^{Q+1}, \mathbf{W}_V \cdot \gamma \rangle$
 - 27: $\mu = \gamma_I x + \gamma_O x^2 + \gamma_S x^3$
 - 28: $\hat{t} = t(x)$
 - 29: $\mathbf{l} = l(x)$
 - 30: $\mathbf{r} = r(x)$
 - 31: $\mathcal{P} \rightarrow \mathcal{V} : \tau_x, \mu, \hat{t}, \mathbf{l}, \mathbf{r}$
 - 32: $\mathcal{V}$:
 - 33: $\hat{t} \stackrel{?}{=} \langle \mathbf{l}, \mathbf{r} \rangle$ check the correctness of $\hat{t}$
 - 34: $\hat{t} G + \tau_x H \stackrel{?}{=} x^2 \delta_{new}(y, z) G + x^2 \langle \mathbf{z}_{[1:]}^{Q+1}, \mathbf{c} \rangle G + x^2 \langle \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_V, \mathbf{V} \rangle$
 - 35: $+xT_1 + \sum_{i=3}^6 x^i T_i$ check the constraints
 - 36: $x A_I + x^2 A_O + x^3 S - \langle \mathbf{1}^n, \mathbf{H} \rangle + x \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L), \mathbf{H} \rangle$

$$\begin{array}{ll}
37: & +x\langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R), \mathbf{G} \rangle + \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_O), \mathbf{H} \rangle \stackrel{?}{=} \\
38: & \mu H + \langle \mathbf{l}, \mathbf{G} \rangle + \langle \mathbf{y}^n \circ \mathbf{r}, \mathbf{H} \rangle \quad \text{check the consistency of commitments}
\end{array}$$

Figure 1: Modified arithmetic circuit interactive argument $(\mathcal{P}, \mathcal{V})$.

Specifically, we run into the aforementioned problem when we attempt to accumulate the verifier check enforcing the constraints, since the term $\delta(y, z)$ maps the verifier challenge y to $\mathbf{y}^{-n}$ in the original argument. We solve this problem by introducing a new algebraic verifier $\mathcal{V}^{\delta_{\text{new}}}$ without negative exponents, obtained by substituting $\mathbf{y}^n \rightarrow l(x)$ and $\mathbf{y}^{-n} \rightarrow r(x)$. In particular, we have that:

$$\begin{aligned}
\langle l(x), r(x) \rangle &= \langle x(\mathbf{a}_L + \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R)) + x^2 \mathbf{a}_O + x^3 \mathbf{s}_L, -\mathbf{y}^{-n} + \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_O \\
&\quad + x(\mathbf{y}^{-n} \circ \mathbf{a}_R + \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L) + x^3(\mathbf{y}^{-n} \circ \mathbf{s}_R) \rangle.
\end{aligned}$$

From this inner product, we derive the term

$$\begin{aligned}
t_2 &= \langle \mathbf{a}_L, \mathbf{y}^{-n} \circ \mathbf{a}_R \rangle - \langle \mathbf{a}_O, \mathbf{y}^{-n} \rangle \\
&\quad + \langle \mathbf{a}_L, \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L \rangle + \langle \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R, \mathbf{a}_R \rangle + \langle \mathbf{a}_O, \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_O \rangle \\
&\quad + \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R), \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L \rangle,
\end{aligned}$$

where $\delta_{\text{new}}(y, z) = \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R), \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L \rangle$ does not contain negative exponents.

Hence, we define the algebraic verifier $\mathcal{V}^{\delta_{\text{new}}}$

$$\begin{aligned}
&\mathcal{V}^{\delta_{\text{new}}}(\mathbf{V}, \mathbf{c}, T_1, T_3, \dots, T_6, \tau_x, \hat{t}, y, z, x) = \\
&\hat{t}G + \tau_x H - x^2 \delta_{\text{new}}(y, z)G - x^2 \langle \mathbf{z}_{[1:]}^{Q+1}, \mathbf{c} \rangle G - x^2 \langle \mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_V, \mathbf{V} \rangle \\
&- xT_1 - \sum_{i=3}^6 x^i T_i = 0,
\end{aligned} \tag{7}$$

in the form of Equation (3), and where $\mathbf{V} = (V_j)_{j=1}^m$ and $\mathbf{c} = (c_i)_{i=1}^Q$ are in pi , $(T_1, T_3, \dots, T_6, \tau_x, \hat{t})$ are in $(\mathbf{m}_i)_{i=1}^{k+1}$ and (y, z, x) in $(r_i)_{i=1}^k$. Note that $G, H, \mathbf{G}, \mathbf{H}, \mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O$ and $\mathbf{W}_V$ from Relation (2) are not included in pi , since the CRS and circuit description parameters are not part of the accumulated instances.

From the previous changes, some adjustments need to be made on the verifier check for the consistency of commitments. Following from the new vectors $\mathbf{l}$ and $\mathbf{r}$ and the updated basis $\mathbf{H}' = \mathbf{y}^n \circ \mathbf{H}$, the updated check becomes

$$\begin{aligned}
&x A_I + x^2 A_O + x^3 S - \langle \mathbf{l}^n, \mathbf{H} \rangle + x \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L), \mathbf{H} \rangle \\
&\quad + x \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R), \mathbf{G} \rangle + \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_O), \mathbf{H} \rangle \stackrel{?}{=} \\
&\mu H + \langle \mathbf{l}, \mathbf{G} \rangle + \langle \mathbf{y}^n \circ \mathbf{r}, \mathbf{H} \rangle,
\end{aligned} \tag{8}$$

expressed entirely in basis $\mathbf{H}$.

The following theorem shows that the modified arithmetic circuit proof of Bulletproofs, presented in Figure 1, preserves security.

Theorem 1 (Modified arithmetic circuit proof (Bulletproofs)). *The modified arithmetic circuit proof of Bulletproofs has perfect completeness, perfect honest-verifier zero-knowledge and special-soundness satisfying Relation (2).*

Proof sketch. The proof proceeds analogously to the Bulletproofs proof. Completeness holds since t_2 is derived correctly and an honest prover conversation is accepted. Zero-knowledge follows by the existence of a simulator that, given the statement and the public-coin randomness, outputs a transcript $(A_I, A_O, S, y, z, T_1, T_3, \dots, T_6, x, \tau_x, \mu, \hat{t}, \mathbf{l}, \mathbf{r})$ that is identically distributed to the transcript produced in an honest execution between prover and verifier. To prove special-soundness, we can use the same extractor of Bulletproofs, which rewinds the prover and processes a poly-bounded (in λ) set of accepting transcripts to extract either a valid witness $(\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O, \mathbf{v}, \gamma)$ or a non-trivial DLOG relation. This concludes the proof sketch.

Following the ProtoStar’s recipe, Equation (7) can be expressed as a sum of homogeneous degree- d algebraic maps $f_d^{\mathcal{V}^{\delta_{\text{new}}}}(\mathbf{V}, \mathbf{c}, T_1, T_3, \dots, T_6, \tau_x, \hat{t}, y, z, x)$, for $0 \leq d \leq n + 1 + 2Q$. The upper bound follows from the maximum polynomial degree contributed by the term $-x^2 \delta_{\text{new}}(y, z)G$. In what follows, we give the full decomposition of $\mathcal{V}^{\delta_{\text{new}}}$ into homogeneous maps with implicit parameters.

$$\begin{aligned} f_0^{\mathcal{V}^{\delta_{\text{new}}}} &= 0, \\ f_1^{\mathcal{V}^{\delta_{\text{new}}}} &= \hat{t}G + \tau_x H, \\ f_2^{\mathcal{V}^{\delta_{\text{new}}}} &= -xT_1, \\ f_3^{\mathcal{V}^{\delta_{\text{new}}}} &= 0, \\ f_{4 \leq d \leq 7}^{\mathcal{V}^{\delta_{\text{new}}}} &= -x^{d-1}T_{d-1} - x^2 z^{d-3} c_{d-3} G \\ &\quad - x^2 \sum_{k=1}^m z^{d-3} w_{V_{d-3,k}} V_k - x^2 \left(\sum_{i=2}^{d-2} k_{i-1,d-i} y^{i-2} z^{d-i} \right) G, \end{aligned}$$

where $k_{j,s} = \sum_{i=1}^Q w_{R_{i,j}} w_{L_{s-i,j}}$ with $w_{L_{s-i,j}} = 0$ if $s-i \notin [1, Q]$. Finally, the maps $f_d^{\mathcal{V}^{\delta_{\text{new}}}}$ for $7 < d \leq Q+3$ follow from the previous but omit the contribution of the terms $-\sum_{i=3}^6 x^i T_i$, while the maps $f_d^{\mathcal{V}^{\delta_{\text{new}}}}$ for $Q+3 < d \leq n+1+2Q$ consider only $-x^2 \delta_{\text{new}}(y, z)G$.

It remains the verifier check that enforces the correctness of $\hat{t}$, which does not require any adjustment due to negative exponents. We write the algebraic verifier $\mathcal{V}^{\hat{t}}$ in the form of Equation (3):

$$\mathcal{V}^{\hat{t}}(\hat{t}, \mathbf{l}, \mathbf{r}) = \hat{t} - \langle \mathbf{l}, \mathbf{r} \rangle = 0 \quad (9)$$

Then, Equation (9) can be expressed as a sum of homogeneous degree- d algebraic maps $f_d^{\mathcal{V}^{\hat{t}}}(\hat{t}, \mathbf{l}, \mathbf{r})$, for $0 \leq d \leq 2$. The full decomposition of $\mathcal{V}^{\hat{t}}$ into homogeneous maps with implicit parameters is: $f_0^{\mathcal{V}^{\hat{t}}} = 0$, $f_1^{\mathcal{V}^{\hat{t}}} = \hat{t}$, $f_2^{\mathcal{V}^{\hat{t}}} = -\langle \mathbf{l}, \mathbf{r} \rangle$.

4.3 Computing the error terms

The prover accumulates "relaxed" verifier checks by computing the error terms via Equation (5). Let $\mathcal{V}$ be an algebraic verifier of degree D , in the form of a sum of homogeneous degree- d maps $f_d^{\mathcal{V}}$, and define interpolated inputs as $u''(X) := Xu + u'$. Here, u can be any element of the statement or of the NARK proof, and u' is the corresponding element of the accumulator. For a slack σ , we have that:

$$\begin{aligned} & \sum_{d=0}^D (X + \sigma)^{D-d} \cdot f_d^{\mathcal{V}}(u''(X)) = \\ & \sum_{d=0}^D \left(\sum_{a=0}^{D-d} \binom{D-d}{a} \sigma^{D-d-a} X^a \right) \cdot \left(\sum_{t=0}^d f_{d,t}^{\mathcal{V}} X^t \right) = \\ & \sum_{j=0}^D \left(\sum_{d=0}^D \sum_{a=0}^{D-d} \binom{D-d}{a} \sigma^{D-d-a} f_{d,j-a}^{\mathcal{V}} \right) X^j, \end{aligned}$$

where $f_{d,j-a}^{\mathcal{V}} = 0$ if $j - a \notin [0, d]$.

It turns out that, for $j = 1, \dots, D-1$, the intermediate-degree coefficients are the error terms

$$\mathbf{e}_j = \sum_{d=0}^D \sum_{a=0}^{D-d} \binom{D-d}{a} \sigma^{D-d-a} f_{d,j-a}^{\mathcal{V}},$$

and the constant coefficient is $\mathbf{e} = \sum_{d=0}^D \sigma^{D-d} f_{d,0}^{\mathcal{V}}$.

In our design, the prover accumulates error terms for the algebraic verifiers $\mathcal{V}^{\delta_{\text{new}}}$ and $\mathcal{V}^{\hat{t}}$ of the NARK, i.e., Equation (7) and Equation (9) respectively⁴. For each algebraic verifiers, the error terms are computed by deriving and collecting the coefficients $f_{d,t}^{\mathcal{V}^{\delta_{\text{new}}}}$ and $f_{d,t}^{\mathcal{V}^{\hat{t}}}$ as in the above procedure. In what follows, we provide formulas for deriving these coefficients, but the exact expressions may vary depending on the implementation.

Consider Equation (7) and let

⁴ Note that the verifier check in Equation (8) is not accumulated, since it is the consistency check of commitments.

$$\begin{aligned}
\mathbf{V}''(X) &= (V_j''(X))_{j=1}^m = (XV_j + V_j')_{j=1}^m, \\
\mathbf{c}''(X) &= (c_i''(X))_{i=1}^Q = (Xc_i + c_i')_{i=1}^Q, \\
T_1''(X) &= XT_1 + T_1', \quad T_3''(X) = XT_3 + T_3', \dots, T_6''(X) = XT_6 + T_6', \\
\tau_x''(X) &= X\tau_x + \tau_x', \quad \hat{t}''(X) = X\hat{t} + \hat{t}', \\
y''(X) &= Xy + y', \quad z''(X) = Xz + z', \quad \text{and } x''(X) = Xx + x'.
\end{aligned}$$

By applying Equation (5) with these inputs, we derive the full list of coefficients $f_{d,t}^{\mathcal{V}^{\delta_{\text{new}}}}$, for $0 \leq d \leq n+1+2Q$ and $0 \leq t \leq d$:

$$\begin{aligned}
f_{0,0}^{\mathcal{V}^{\delta_{\text{new}}}} &= 0, \\
f_{1,0}^{\mathcal{V}^{\delta_{\text{new}}}} &= \hat{t}'G + \tau_x'H, \quad f_{1,1}^{\mathcal{V}^{\delta_{\text{new}}}} = \hat{t}G + \tau_xH, \\
f_{2,0}^{\mathcal{V}^{\delta_{\text{new}}}} &= -x'T_1, \quad f_{2,1}^{\mathcal{V}^{\delta_{\text{new}}}} = -(xT_1' + x'T_1), \quad f_{2,2}^{\mathcal{V}^{\delta_{\text{new}}}} = -xT_1, \\
f_{3,t}^{\mathcal{V}^{\delta_{\text{new}}}} &= 0, \\
f_{(4 \leq d \leq 7),t}^{\mathcal{V}^{\delta_{\text{new}}}} &= -\binom{d-1}{t-1} x^{t-1} (x')^{d-t} T_{d-1} - \binom{d-1}{t} x^t (x')^{d-1-t} T_{d-1}' \\
&\quad - \sum_{\substack{u \in [0,2] \\ v \in [0,d-3] \\ w \in \{0,1\} \\ u+v+w=t}} \binom{2}{u} \binom{d-3}{v} x^u (x')^{2-u} z^v (z')^{d-3-v} (c_{d-3})^w (c_{d-3}')^{1-w} G \\
&\quad - \sum_{k=1}^m w_{V_{d-3,k}} \sum_{\substack{u \in [0,2] \\ v \in [0,d-3] \\ w \in \{0,1\} \\ u+v+w=t}} \binom{2}{u} \binom{d-3}{v} x^u (x')^{2-u} z^v (z')^{d-3-v} V_k^w (V_k')^{1-w} \\
&\quad - \left(\sum_{j=2}^{d-2} k_{j-1,d-j} \sum_{\substack{u \in [0,2] \\ v \in [0,j-2] \\ w \in [0,d-j] \\ u+v+w=t}} \binom{2}{u} \binom{j-2}{v} \binom{d-j}{w} x^u (x')^{2-u} y^v (y')^{j-2-v} z^w (z')^{d-j-w} \right) G,
\end{aligned}$$

and $f_{d,t}^{\mathcal{V}^{\delta_{\text{new}}}}$ for $7 < d \leq Q+3$ follow from the previous but without the folded terms of $-\sum_{i=3}^6 x^i T_i$, while $f_{d,t}^{\mathcal{V}^{\delta_{\text{new}}}}$ for $Q+3 < d \leq n+1+2Q$ consider only the folded terms of $-x^2 \delta_{\text{new}}(y, z)G$.

Finally, consider Equation (9) and let

$$\hat{t}''(X) = X\hat{t} + \hat{t}', \quad \mathbf{l}''(X) = X\mathbf{l} + \mathbf{l}', \quad \mathbf{r}''(X) = X\mathbf{r} + \mathbf{r}'.$$

By applying Equation (5) with these inputs, we derive the full list of coefficients $f_{d,t}^{\mathcal{V}^i}$, for $0 \leq d \leq 2$ and $0 \leq t \leq d$:

$$\begin{aligned} f_{0,0}^{\mathcal{V}^i} &= 0 \\ f_{1,0}^{\mathcal{V}^i} &= \hat{t}', \quad f_{1,1}^{\mathcal{V}^i} = \hat{t}, \\ f_{2,0}^{\mathcal{V}^i} &= -\langle \mathbf{l}', \mathbf{r}' \rangle, \quad f_{2,1}^{\mathcal{V}^i} = -\langle \mathbf{l}, \mathbf{r}' \rangle - \langle \mathbf{l}', \mathbf{r} \rangle, \quad f_{2,2}^{\mathcal{V}^i} = -\langle \mathbf{l}, \mathbf{r} \rangle. \end{aligned}$$

5 Bulletproofs* protocols

5.1 Folding scheme for arithmetic circuits

We present a folding scheme $(\mathcal{P}_{fold}, \mathcal{V}_{fold}, \mathcal{D})$ for Bulletproofs, which we call *Bulletproofs**. To this end, we consider the NARK of the modified arithmetic circuit proof presented in Figure 1, which adopts the algebraic verifiers $\mathcal{V}^{\delta_{new}}$ and $\mathcal{V}^i$ introduced in Section 4.2. We instantiate two random oracles, $\mathcal{O}_{NARK}$ and $\mathcal{O}_{fold}$, for our scheme. All algorithms share the CRS and circuit description parameters from Relation (2). For ease of exposition, we treat these parameters as implicit inputs to the folding scheme.

Folding prover $\mathcal{P}_{fold}$. On input the statement $\text{pi} = \{\mathbf{V}, \mathbf{c}\}$, the NARK proof π , and the accumulator acc , where

$$\begin{aligned} \pi &= \{\pi_x = \{A_I, A_O, S, T_1, T_3, \dots, T_6\}, \pi_w = \{\tau_x, \mu, \hat{t}, \mathbf{l}, \mathbf{r}\}\}, \\ \text{acc} &= \{\text{acc}_x = \{\mathbf{V}', \mathbf{c}', A'_I, A'_O, S', T'_1, T'_3, \dots, T'_6, y', z', x', E^{(1)}, E^{(2)}, \sigma\}, \\ &\quad \text{acc}_w = \{\tau'_x, \mu', \hat{t}', \mathbf{l}', \mathbf{r}'\}\}, \end{aligned}$$

the folding prover $\mathcal{P}_{fold}$ works as in Figure 2.

Folding verifier $\mathcal{V}_{fold}$. On input the statement pi , the NARK proof instance π_x , the accumulator instance acc_x , the updated accumulator instance

$$\text{acc}'_x = \{\mathbf{V}'', \mathbf{c}'', A''_I, A''_O, S'', T''_1, T''_3, \dots, T''_6, y'', z'', x'', E'^{(1)}, E'^{(2)}, \sigma'\},$$

and the accumulation proof pf , the folding verifier $\mathcal{V}_{fold}$ works as in Figure 3.

Decider $\mathcal{D}$. On input an accumulator

$$\begin{aligned} \text{acc} &= \{\text{acc}_x = \{\mathbf{V}, \mathbf{c}, A_I, A_O, S, T_1, T_3, \dots, T_6, y, z, x, E^{(1)}, E^{(2)}, \sigma\}, \\ &\quad \text{acc}_w = \{\tau_x, \mu, \hat{t}, \mathbf{l}, \mathbf{r}\}\}, \end{aligned}$$

the decider $\mathcal{D}$ works as in Figure 4.

1: **input:** ($\text{pi}, \pi, \text{acc}$)

2: **output:** $\{\text{acc}', \text{pf}\}$

3: $y \leftarrow \mathcal{O}_{NARK}(V_1 || \dots || V_m || c_1 || \dots || c_Q || A_I || A_O || S)$

4: $z \leftarrow \mathcal{O}_{NARK}(y)$

5: $x \leftarrow \mathcal{O}_{NARK}(z || T_1 || T_3 || T_4 || T_5 || T_6)$

6: $\mathbf{V}''(X) = (V_j''(X))_{j=1}^m = (XV_j + V_j')_{j=1}^m$

7: $\mathbf{c}''(X) = (c_i''(X))_{i=1}^Q = (Xc_i + c_i')_{i=1}^Q$

8: $A_I''(X) = XA_I + A_I', A_O''(X) = XA_O + A_O', S''(X) = XS + S'$

9: $T_1''(X) = XT_1 + T_1', T_3''(X) = XT_3 + T_3', \dots, T_6''(X) = XT_6 + T_6'$

10: $\tau_x''(X) = X\tau_x + \tau_x', \mu''(X) = X\mu + \mu', \hat{t}''(X) = X\hat{t} + \hat{t}'$

11: $\mathbf{l}''(X) = X\mathbf{l} + \mathbf{l}', \mathbf{r}''(X) = X\mathbf{r} + \mathbf{r}'$

12: $y''(X) = Xy + y', z''(X) = Xz + z', x''(X) = Xx + x'$

13: $e^{(1)}(X) = \mathbf{e}^{(1)} + \sum_{j=1}^{D-1} \mathbf{e}_j^{(1)} X^j \quad // \sum_{d=0}^D (X + \sigma)^{D-d} \cdot f_d^{\mathcal{V}^{\delta_{\text{new}}}}(u''(X))$

14: $e^{(2)}(X) = \mathbf{e}^{(2)} + \mathbf{e}_1^{(2)} X \quad // \sum_{d=0}^2 (X + \sigma)^{2-d} \cdot f_d^{\mathcal{V}^{\hat{t}}}(u''(X))$

15: $\alpha \leftarrow \mathcal{O}_{fold}(\text{acc}_x || \text{pi} || \pi_x || \mathbf{e}_1^{(1)} || \dots || \mathbf{e}_{D-1}^{(1)} || \mathbf{e}_1^{(2)})$

16: $E'^{(1)} = E^{(1)} + e^{(1)}(\alpha) - e^{(1)}(0)$

17: $E'^{(2)} = E^{(2)} + e^{(2)}(\alpha) - e^{(2)}(0)$

18: $\text{acc}' = \{\mathbf{V}''(\alpha), \mathbf{c}''(\alpha), A_I''(\alpha), A_O''(\alpha), S''(\alpha), T_1''(\alpha), T_3''(\alpha), \dots, T_6''(\alpha), y''(\alpha),$

19: $z''(\alpha), x''(\alpha), E'^{(1)}, E'^{(2)}, \sigma' = \alpha + \sigma; \tau_x''(\alpha), \mu''(\alpha), \hat{t}''(\alpha), \mathbf{l}''(\alpha), \mathbf{r}''(\alpha)\}$

20: $\text{pf} = \{\mathbf{e}_1^{(1)}, \dots, \mathbf{e}_{D-1}^{(1)}, \mathbf{e}_1^{(2)}\}$

Figure 2: Folding Prover $\mathcal{P}_{\text{fold}}^{\mathcal{O}_{NARK}, \mathcal{O}_{fold}}$.

1: **input:** $(\text{pi}, \pi_x, \text{acc}_x, \text{acc}'_x, \text{pf})$

2: **output:** $\{0, 1\}$

3: $y \leftarrow \mathcal{O}_{NARK}(V_1 || \dots || V_m || c_1 || \dots || c_Q || A_I || A_O || S)$

4: $z \leftarrow \mathcal{O}_{NARK}(y)$

5: $x \leftarrow \mathcal{O}_{NARK}(z || T_1 || T_3 || T_4 || T_5 || T_6)$

6: $\alpha \leftarrow \mathcal{O}_{fold}(\text{acc}_x || \text{pi} || \pi_x || \mathbf{e}_1^{(1)} || \dots || \mathbf{e}_{D-1}^{(1)} || \mathbf{e}_1^{(2)})$

7: $(V_j'')_{j=1}^m \stackrel{?}{=} (\alpha \cdot V_j + V_j')_{j=1}^m, (c_i'')_{i=1}^Q \stackrel{?}{=} (\alpha \cdot c_i + c_i')_{i=1}^Q$

8: $y'' \stackrel{?}{=} \alpha \cdot y + y', z'' \stackrel{?}{=} \alpha \cdot z + z', x'' \stackrel{?}{=} \alpha \cdot x + x'$

9: $A_I'' \stackrel{?}{=} \alpha \cdot A_I + A_I', A_O'' \stackrel{?}{=} \alpha \cdot A_O + A_O', S'' \stackrel{?}{=} \alpha \cdot S + S'$

10: $T_1'' \stackrel{?}{=} \alpha \cdot T_1 + T_1', (T_i'')_{i=3}^6 \stackrel{?}{=} (\alpha \cdot T_i + T_i')_{i=3}^6, \sigma' \stackrel{?}{=} \alpha + \sigma$

11: $E'^{(1)} \stackrel{?}{=} E^{(1)} + \sum_{j=1}^{D-1} \alpha^j \cdot \mathbf{e}_j^{(1)}$

12: $E'^{(2)} \stackrel{?}{=} E^{(2)} + \alpha \cdot \mathbf{e}_1^{(2)}$

Figure 3: Folding Verifier $\mathcal{V}_{\text{fold}}^{\mathcal{O}_{NARK}, \mathcal{O}_{fold}}$.

```

1: input: (acc)
2: output:  $\{0, 1\}$ 
3:  $\mathbf{z}_{[1:]}^{Q+1} = (z, z^2, \dots, z^Q)$ 
4:  $\mathbf{y}^n = (1, y, y^2, \dots, y^{n-1})$ 
5:  $\mathbf{r}' = \mathbf{y}^n \circ \mathbf{r}$ 
6:  $W_L = \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_L), \mathbf{H} \rangle$ 
7:  $W_R = \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_R), \mathbf{G} \rangle$ 
8:  $W_O = \langle \mathbf{y}^n \circ (\mathbf{z}_{[1:]}^{Q+1} \cdot \mathbf{W}_O), \mathbf{H} \rangle$ 
9:  $\text{Commit}(\mathbf{l}, \mathbf{r}', \mu) \stackrel{?}{=} xA_I + x^2A_O + x^3S - \langle \mathbf{1}^n, \mathbf{H} \rangle + xW_L + xW_R + W_O$ 
10:  $E^{(1)} \stackrel{?}{=} \sum_{d=0}^D \sigma^{D-d} \cdot f_d^{\mathcal{V}^{\delta_{\text{new}}}}(\mathbf{V}, \mathbf{c}, T_1, T_3, \dots, T_6, \tau_x, \hat{t}, y, z, x) \ // \ D = n + 1 + 2Q$ 
11:  $E^{(2)} \stackrel{?}{=} \sum_{d=0}^2 \sigma^{2-d} \cdot f_d^{\mathcal{V}^{\hat{t}}}(\hat{t}, \mathbf{l}, \mathbf{r})$ 

```

Figure 4: Decider $\mathcal{D}$.

The following theorem shows that the folding scheme $(\mathcal{P}_{fold}, \mathcal{V}_{fold}, \mathcal{D})$ is secure.

Theorem 2 (Bulletproofs* folding scheme). *The folding scheme $(\mathcal{P}_{fold}, \mathcal{V}_{fold}, \mathcal{D})$ has perfect completeness and knowledge soundness.*

Proof sketch. Completeness and knowledge soundness follow the same proof of ProtoStar. In order to prove completeness, let $(\text{pi}, \pi, \text{acc}) \in \mathcal{R}_{fold}$, that is, $\mathcal{V}_{NARK}(\text{pi}, \pi) = 1$ and $\mathcal{D}(\text{acc}) = 1$. Then, $\mathcal{D}(\text{acc}') = 1$ and $\mathcal{V}_{fold}(\text{pi}, \pi_x, \text{acc}_x, \text{acc}'_x, \text{pf}) = 1$, where (acc', pf) is the output of $\mathcal{P}_{fold}(\text{pi}, \pi, \text{acc})$. This holds if the linear combinations through α are consistent in $\mathcal{P}_{fold}$ and $\mathcal{V}_{fold}$, the commitment scheme is homomorphic, and the accumulation proof is computed correctly. Knowledge soundness follows by the existence of a special-sound (public-coin) interactive protocol and by applying the Fiat-Shamir transform to it. The interactive protocol $(\mathcal{P}, \mathcal{V})$ is defined as follows. $\mathcal{P}$ sends the accumulation proof pf , $\mathcal{V}$ replies with a random challenge α , then $\mathcal{P}$ responds with acc' . The verifier $\mathcal{V}$ accepts if $\mathcal{D}(\text{acc}') = 1$ and $\mathcal{V}_{fold}(\text{pi}, \pi_x, \text{acc}_x, \text{acc}'_x, \text{pf}) = 1$ using the random challenge α . This concludes the proof sketch.

5.2 Complexity of the folding scheme

We provide the computational complexity of the folding scheme $(\mathcal{P}_{fold}, \mathcal{V}_{fold}, \mathcal{D})$; the dimensions n, m and Q are specified in Relation (2), and MSM denotes a multi-scalar multiplication.

The folding prover $\mathcal{P}_{fold}$:

- does 3 queries to $\mathcal{O}_{NARK}$ and 1 query to $\mathcal{O}_{fold}$;
- computes the coefficients of a degree $D = n + 1 + 2Q$ polynomial for $\mathbf{e}_1^{(1)}, \dots, \mathbf{e}_{D-1}^{(1)}$, and the coefficients of a degree 2 polynomial for $\mathbf{e}_1^{(2)}$;
- 1 MSM of size $D - 1 = n + 2Q$ plus 1 group addition to compute $E'^{(1)}$;

- 2 field operations to compute $E'^{(2)}$;
- $m + 8$ scalar multiplications plus $m + 8$ group addition to compute $\mathbf{V}'', A_I'', A_O'', S'', T_1'', T_3'', \dots, T_6''$;
- $2Q + 4n + 13$ field operations to compute $\mathbf{c}'', y'', z'', x'', \sigma', \tau_x'', \mu'', \hat{t}'', \mathbf{l}'', \mathbf{r}''$.

The folding verifier $\mathcal{V}_{fold}$:

- does 3 queries to $\mathcal{O}_{NARK}$ and 1 query to $\mathcal{O}_{fold}$;
- $m + 8$ scalar multiplications plus $m + 8$ group addition to check $\mathbf{V}'', A_I'', A_O'', S'', T_1'', T_3'', \dots, T_6''$;
- $2Q + 7$ field operations to check $\mathbf{c}'', y'', z'', x'', \sigma'$;
- 1 MSM of size $D - 1 = n + 2Q$ plus 1 group addition to check $E'^{(1)}$;
- 2 field operations to check $E'^{(2)}$.

The decider $\mathcal{D}$:

- $6nQ + 2n + Q$ field operations to compute $\mathbf{y}^n, \mathbf{z}_{[1:]}^{Q+1}, \mathbf{r}'$ and the Hadamard products for $\mathbf{W}_{L,R,O}$;
- 3 MSM of size n to compute $W_{L,R,O}$;
- checks the consistency of commitments with 2 MSM of size n plus 1 scalar multiplication plus 2 group addition to compute $\text{Commit}(\mathbf{1}, \mathbf{r}', \mu)$, and 3 field operations plus 5 scalar multiplications plus 6 group additions to compute the right-hand side of the consistency check;
- checks $E^{(1)}$ by evaluating the relaxed verifier $V^{\delta_{new}}$ of degree $D = n + 1 + 2Q$;
- checks $E^{(2)}$ by evaluating the relaxed verifier $V^{\hat{t}}$ of degree $D = 2$.

6 Comparison

By Theorem 3 of ProtoStar, we can evaluate the performance of our folding scheme after the folding-to-IVC transformation. To show that this compilation is non-trivial, we estimate the runtime of the resulting IVC verifier and compare it with the cost of the original monolithic Bulletproofs verifier over N invocations. Consider the interactive protocol of the modified Bulletproofs in Figure 1, i.e., a 5-move special-sound interactive protocol with $k + 1$ prover message rounds and $k = 2$ verifier challenge rounds. Moreover, the interactive verifier checks two algebraic maps of output length 1 each. Then, there exists an IVC scheme $(\mathcal{P}_{IVC}, \mathcal{V}_{IVC})$ and $\mathcal{V}_{IVC}$ has the following efficiency:

$$2 + \sum_{i=1}^3 |m_i| \mathbb{G} + \mathcal{V}_{BP*}, \quad (10)$$

where the quantity 2 is related to the total output length l of the algebraic maps, $|m_i|$ is the prover message length, $\mathbb{G}$ the total length of the committed messages, and $\mathcal{V}_{BP*}$ the cost of running the algebraic verifier of the interactive protocol. The quantity in (10) compares with the cost of verifying N proofs via the monolithic Bulletproofs verifier, which is approximately $N \cdot \mathcal{V}_{BP}$. Hence, verifying folded proofs is asymptotically more efficient than verifying N proofs via a monolithic Bulletproofs verifier, with a gain that scales linearly in N .

7 Conclusions and future directions

In this work, we presented Bulletproofs*, a folding scheme for arithmetic circuit proofs under standard assumptions and without preprocessing. To this end, we first adapted the algebraic verifiers of the arithmetic circuit proof of Bulletproofs to the algebraic form required by ProtoStar, and proved that the modified protocol remains secure. Then, we designed the Bulletproofs folding scheme that is complete and knowledge-sound. Finally, we analyzed the resulting verifier cost after the folding-to-IVC transformation. The result shows an asymptotic linear gain compared to repeated invocations of the monolithic Bulletproofs verifier. In the current design, the recursive circuit is linear in the degree of the algebraic verifier, and hence the arithmetic circuit complexity. This may be optimized by reducing the number of group operations using the ProtoStar compression technique, i.e., by compressing verifier checks via a random linear combination. Moreover, assuming constant-sized statements, the accumulator witness size is also linear in the circuit complexity; introducing an IPA could reduce it to logarithmic size. We leave these optimizations to future work, together with an implementation for concrete performance evaluation and an extended comparison.

References

1. Alonso, K.M., et al.: Zero to monero (2020)
2. Attema, T., Cramer, R.: Compressed-protocol theory and practical application to plug & play secure algorithmics. In: Annual International Cryptology Conference. pp. 513–543. Springer (2020)
3. Attema, T., Fehr, S., Klooß, M.: Fiat-shamir transformation of multi-round interactive proofs. In: Theory of Cryptography Conference. pp. 113–142. Springer (2022)
4. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: Proceedings of the 1st ACM Conference on Computer and Communications Security. pp. 62–73 (1993)
5. Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. *Algorithmica* **79**(4), 1102–1160 (2017)
6. Boneh, D., Drake, J., Fisch, B., Gabizon, A.: Halo infinite: Proof-carrying data from additive polynomial commitments. In: Annual International Cryptology Conference. pp. 649–680. Springer (2021)
7. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 327–357. Springer (2016)
8. Bowe, S., Grigg, J., Hopwood, D.: Recursive proof composition without a trusted setup. *Cryptology ePrint Archive* (2019)
9. Bünz, B., Agrawal, S., Zamani, M., Boneh, D.: Zether: Towards privacy in a smart contract world. In: International Conference on Financial Cryptography and Data Security. pp. 423–443. Springer (2020)

10. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE symposium on security and privacy (SP). pp. 315–334. IEEE (2018)
11. Bünz, B., Chen, B.: Protostar: Generic efficient accumulation/folding for special-sound protocols. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 77–110. Springer (2023)
12. Bünz, B., Chiesa, A., Lin, W., Mishra, P., Spooner, N.: Proof-carrying data without succinct arguments. In: Annual International Cryptology Conference. pp. 681–710. Springer (2021)
13. Bünz, B., Chiesa, A., Mishra, P., Spooner, N.: Proof-carrying data from accumulation schemes. Cryptology ePrint Archive (2020)
14. Chung, H., Han, K., Ju, C., Kim, M., Seo, J.H.: Bulletproofs+: Shorter proofs for a privacy-enhanced distributed ledger. *Ieee Access* **10**, 42081–42096 (2022)
15. Eagen, L., Kanjalkar, S., Ruffing, T., Nick, J.: Bulletproofs++: next generation confidential transactions via reciprocal set membership arguments. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 249–279. Springer (2024)
16. Fauzi, P., Meiklejohn, S., Mercer, R., Orlandi, C.: Quisquis: A new design for anonymous cryptocurrencies. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 649–678. Springer (2019)
17. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986)
18. Kothapalli, A., Setty, S., Tzalla, I.: Nova: Recursive zero-knowledge arguments from folding schemes. In: Annual International Cryptology Conference. pp. 359–388. Springer (2022)
19. Lindell: Parallel coin-tossing and constant-round secure two-party computation. *Journal of Cryptology* **16**, 143–184 (2003)
20. Maller, M., Bowe, S., Kohlweiss, M., Meiklejohn, S.: Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings. In: Proceedings of the 2019 ACM SIGSAC conference on computer and communications security. pp. 2111–2128 (2019)
21. Scala, E.: Trustless zero-knowledge proofs in public blockchain (2024)
22. Scala, E., Dong, C., Corradini, F., Mostarda, L.: Zeromt: Towards multi-transfer transactions with privacy for account-based blockchain. *Journal of Information Security and Applications* **83**, 103794 (2024)
23. Scala, E., Mostarda, L.: Efficient inner-product argument from compressed σ -protocols and applications. In: International Conference on Advanced Information Networking and Applications. pp. 326–337. Springer (2024)
24. Thaler, J.: Proofs, arguments, and zero-knowledge. *Foundations and Trends® in Privacy and Security* **4**(2-4), 117–660 (2022)